

## Vulnerability Assessment : Simulated FinTech Web Application

Includes:

1. Goals and Objectives
2. Scope
3. Information Gathering
4. Vulnerability Detection
5. Information analysis and planning

**Target:** A practice web app (`10.0.2.8`) running inside an isolated VirtualBox + Kali Linux lab.

**Tools used:** Nmap, Gobuster, dirb, wordlists, Kali Linux, Burp Suite, Virtual Box

### Quick Summary :

I found six issues. As a beginner, the biggest thing I learned is that they're almost all the same kind of problem: the app is leaking sensitive information in places it shouldn't old debug pages, an open FTP service, comments left in the code, and traffic sent without encryption. Most of these come down to security misconfiguration, which basically means stuff that was left turned on or exposed by mistake.

In a real finance app, these are exactly the kinds of mistakes that leak people's personal and financial data, so the pattern itself felt like the real lesson, not just each individual flag.

**A quick word on the labels I use below, since I had to look these up myself:**

CWE = a standard ID for a type of weakness (like a catalog number).

OWASP Top 10 = the industry's list of the 10 most common web security risks.

CVSS = a 0–10 score for how bad a vulnerability is. I calculated these as estimates while learning, so treat them as rough.

| S.No | What I found                                               | CWE             | OWASP Top 10 | severity estimate |
|------|------------------------------------------------------------|-----------------|--------------|-------------------|
| 1    | FTP let me grab files (including a key file) with no login | CWE287 / CWE319 | A05 / A07    | High (7.5)        |
| 2    | 'robots.txt' had a secret sitting in it                    | CWE200          | A05          | Medium (5.3)      |
| 3    | A secret was left inside an HTML comment                   | CWE615          | A05          | Medium (5.3)      |
| 4    | Same comment problem, but on the admin login page          | CWE615 / CWE200 | A05 / A01    | Medium (6.5)      |
| 5    | A 'phpinfo()' debug page exposed the whole server set      | CWE200 / CWE489 | A05 / A06    | Medium (5.3)      |

|   |                                                         |        |     |              |
|---|---------------------------------------------------------|--------|-----|--------------|
| 6 | Sensitive data was sent over plain HTTP (no encryption) | CWE319 | A02 | Medium (6.1) |
|---|---------------------------------------------------------|--------|-----|--------------|

### Finding 1: I could read files over FTP without logging in

**Insecure FTP** : sensitive files exposed without any authentication

CWE: CWE287 (no real login check), CWE319 (data sent unencrypted), CWE200 (information leak)

OWASP: A05 Security Misconfiguration; A07 Identification & Authentication Failures

Severity: High — `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` (7.5)

**Exploit:** I connected to the FTP service and was able to list and download files without any password. I found a folder with `flag1.txt` in it, and “cat” ing the file gave me the flag. While listing the directory I also noticed a file called `archivekey.asc`, which I learned is a cryptographic key file (a PGP key).

**significance:** Anyone could just walk in and read files off the server. The key file worried me the most from what I read, if a private key leaks, an attacker could potentially decrypt protected data or pretend to be the server. In a finance app that's a straight line to leaking real customer data.

#### Mitigation:

Turn off anonymous FTP so a login is actually required.

Use an encrypted version instead (SFTP or FTPS) so files aren't sent in the clear.

Don't keep sensitive files (keys, backups, flags) in folders that FTP serves.

Lock the service down to trusted networks only.

Because the key was exposed, it should be replaced — once a key is seen, you have to assume it's compromised.

### Finding 2: There was a secret hiding in `robots.txt`.

Sensitive information disclosure through `robots.txt`

CWE: CWE200 (information leak)

OWASP: A05 Security Misconfiguration

**Severity: Medium:** While running Gobuster to bruteforce directories, it flagged `robots.txt`. I opened it in the browser and there was a flag written right inside, next to a `Disallow: /` line.

**significance:** I learned that `robots.txt` is just a polite suggestion for search engines — it's not a security control, and anybody can read it. So, putting a secret in there means it's basically public. On top of that, listing "hidden" folders in `robots.txt` can accidentally hand attackers a map of the things you most wanted to hide.

### **Mitigation:**

Never store secrets in `robots.txt`. Don't use `robots.txt` to "protect" anything use real login/permission checks on those folders keep `robots.txt` for genuinely public crawl rules only.

### **Finding 3: A secret was left in an HTML comment**

Sensitive information left in source code comments

CWE: CWE615 (secrets in comments)

OWASP: A05 Security Misconfiguration

Severity: Medium — `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N` (5.3)

**Exploit:** I right clicked a blog page and used "view source" (view source:). Scrolling through, I found a flag sitting inside an HTML comment, like ``<! flag 3 > ... >``.

**significance:** This was an eyeopener: anything in the page source is visible to every visitor comment included. Developers sometimes leave notes, secrets, or hidden links in comments thinking nobody looks, but anyone can.

### **Mitigation:**

Strip comments out automatically when building for production.

Review code and scan for accidental secrets before going live.

Just never put secrets in frontend code or comments in the first place.

### **Finding 4: Same comment problem, but on the admin login page**

Secret in a comment, plus an admin login page that's wide open to find

CWE: CWE615 (secret in comment), CWE200 (information leak)

OWASP: A05 Security Misconfiguration; A01 Broken Access Control

Severity: Medium

**Exploit:** I found an admin login page at `/4dm1n/` and viewed its source. There was another flag tucked inside an HTML comment. The folder name `4dm1n` looks like they tried to hide it by spelling it weirdly, but it still showed up easily.

**significance:** I rated this a bit higher than the other comment issue because it's two problems together: a leaked secret in the source, and an admin login page anyone can reach. Admin pages are a prime target that's where attackers try password guessing and the loginbypass tricks the rest of the project is about. Trying to hide it with a funny spelling isn't real security; I found it in seconds.

### **Mitigation**

Remove the secret from the admin page source.

Actually, restrict the admin page limit it to specific IPs, a VPN, or an internal network instead of relying on a hard to guess name.

Add protections to admin login like rate limiting, lockout after failed tries, and multifactor authentication.

#### **Finding 5: A debug page exposed the whole server setup**

An exposed phpinfo() page leaking server configuration

CWE: CWE200 (information leak), CWE489 (debug code left in production)

OWASP: A05 Security Misconfiguration; A06 Vulnerable & Outdated Components

Severity: Medium

**Exploit:** At /c0nf1g/ there was a phpinfo() page basically a full diagnostics dump. It showed the PHP version, the operating system, the server type, file paths, and which modules and config files were loaded. I noticed it said PHP 7.4.30, and when I looked it up, that version is endoflife, meaning it no longer gets security patches.

**significance:** A page like this is a gift to an attacker knowing exact versions and file paths lets them look up known exploits for that specific setup. And since the PHP version is outdated and unsupported, that's a second problem on top: running software that may already have public vulnerabilities.

#### **Mitigation:**

Delete phpinfo() and any other debug pages from production.

Hide version banners (e.g. set `expose\_php = Off`).

Update PHP to a current, supported version.

If any diagnostic page is truly needed, put it behind a login and restrict it by IP.

#### **Finding 6 — Sensitive data was being sent without encryption**

Sensitive information sent over plain HTTP (no HTTPS)

CWE: CWE319 (cleartext transmission), CWE200

OWASP: A02 Cryptographic Failures

**Severity: Medium — `CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N` (6.1)**

**Exploit:** Using Burp Suite's Repeater, I looked at a request/response and saw it was a `POST` over plain HTTP/1.1 no TLS, no padlock — with a flag visible in the traffic. So, the data was traveling completely unencrypted.

**significance:** I learned that without HTTPS, anyone able to watch the network (like on shared Wi-Fi, or via a man-in-the-middle attack) can read the whole request and response. In a real finance app that could mean capturing passwords, session cookies, and account details straight off the wire.

#### **Mitigation:**

Use HTTPS/TLS everywhere and automatically redirect HTTP to HTTPS.

Turn on HSTS so browsers refuse to fall back to plain HTTP.

Don't send sensitive values in responses, keep them out of logs, and minimize what's in requests.

Use a modern TLS setup (TLS 1.2 or higher).

1. Most of these are the same root problem. Five of the six come down to security misconfiguration / information leaks. So the real fix isn't six separate patches it's a cleaner release process that strips debug pages, comments, and diagnostic endpoints before anything goes live.
2. Hiding things isn't securing them. The weird folder names (4dm1n, c0nf1g) didn't slow me down at all Gobuster and a bit of guessing found them fast. "Hard to guess" is not the same as "protected."
3. Secrets kept showing up everywhere in FTP, in comments, in `robots.txt`, in traffic. That mirrors how real keys and personal data leak, and a tool that scans for secrets before release would have caught most of them.
4. Encryption was missing in two places FTP files at rest and HTTP traffic in transit so data was exposed both sitting on the server and while moving across the network.

**Recommendations:**

1. Right away: turn off anonymous FTP, replace the exposed key, delete the phpinfo() page, and switch the site to HTTPS.
2. Soon after: strip comments and debug leftovers out of the build, and lock down the admin page.
3. Going forward: add secret scanning and config checks to the release pipeline, update the outdated PHP, and retest to confirm the fixes worked.